Отчёт по домашней работе № 4

«Apache ZooKeeper»

Морозов Дмитрий Александрович, ИСУ 466761

4 ноября 2026 г.

1 Развёртывание ансамбля ZooKeeper

Для работы поднял три сервера **Apache ZooKeeper 3.9.6** в Docker на одном компьютере. Внутри образа используется Java 17: `OpenJDK 17.0.21, Temurin-17.0.21+9`. Каждому серверу выделены отдельные тома для `/data` и `/datalog`, свой файл `myid` с номером 1, 2 или 3. Список участников ансамбля во всех трёх конфигурациях одинаковый.

Для подключения с Windows опубликованы порты 2181, 2182 и 2183. Внутри каждого контейнера клиентский порт равен 2181, порт обмена данными между серверами - 2888, порт выбора лидера - 3888. Совпадение внутренних портов допустимо, поскольку контейнеры имеют разные сетевые адреса. Встроенный AdminServer отключён.

Конфигурация первого сервера:

```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/data
dataLogDir=/datalog
clientPort=2181
minSessionTimeout=4000
maxSessionTimeout=40000
standaloneEnabled=false
reconfigEnabled=false
readonlymode.enabled=false
admin.enableServer=false
4lw.commands.whitelist=srvr,stat,ruok,conf,mntr
server.1=zoo1:2888:3888
server.2=zoo2:2888:3888
server.3=zoo3:2888:3888
```

Конфигурации `zoo2.cfg` и `zoo3.cfg` содержат тот же список `server.N`. Файлы `myid` копируются в записываемые тома перед штатным запуском `zkServer.sh start-foreground`. Параметр сессии координаторов - **10 секунд**; серверные границы `minSessionTimeout=4000` и `maxSessionTimeout=40000` позволяют согласовать это значение.

После первоначального запуска `zoo3` стал лидером, `zoo1` и `zoo2` - последователями. Проверял роли командой `srvr` и выполнением записи/чтения через клиент. Установка всех серверов на одном ПК моделирует отказ отдельных процессов, но не отказ всего компьютера.

2 Проверка кворума и восстановления

Проверки выполнялись последовательно: все три сервера работают; остановлен `zoo3`; дополнительно остановлен `zoo2`; оба сервера запущены снова. На каждом шаге сохранены роли, `docker compose ps -a` и результат реальной клиентской операции.

Состояние	zoo1	zoo2	zoo3	Запись/чтение
3 из 3	follower	follower	leader	Успех
2 из 3	follower	leader	Остановлен	Успех
1 из 3	Не обслуживает	Остановлен	Остановлен	Тайм-аут клиента
Восстановление	follower	leader	follower	Успех

После остановки `zoo3`, который первоначально был лидером, лидером стал `zoo2`. Два оставшихся участника продолжили подтверждать записи. После остановки второго сервера `zoo1` сообщил, что не обслуживает запросы:

```
This ZooKeeper instance is not currently serving requests
```

Клиент не смог подключиться и выполнить операцию:

```
{
  "expected": "unavailable",
  "successful": false,
  "error": "KazooTimeoutError",
  "detail": "Connection time-out",
  "elapsed_seconds": 6.26600000000326,
  "expectation_met": true
}
```

В этом состоянии `docker compose ps -a` показывал `zoo2` и `zoo3` остановленными с кодом 143, а `zoo1` продолжал работать как процесс. Поэтому отсутствие кворума проверялось клиентской операцией, а не только доступностью процесса. После возврата двух серверов снова сформировались один лидер и два последователя, запись и чтение прошли успешно.

Для ансамбля из трёх голосующих участников большинство равно двум. При потере одного большинства достаточно, при потере двух - нет. Полные состояния и результаты клиентских проверок сохранены в каталоге `lab4/evidence/ensemble/20261104T142708Z/`.

3 Эфемерный узел и время жизни сессии

Открыл два независимых клиента `zkCli`. В клиенте A создал постоянный родительский узел и эфемерный дочерний:

```
create /lab4-ephemeral-demo persistent-parent
create -e /lab4-ephemeral-demo/ephemeral-20261104-1 temporary
get -s /lab4-ephemeral-demo/ephemeral-20261104-1
```

В клиенте B выполнил `get -s` для того же дочернего узла. Он существовал и содержал значение `temporary`. Поле `ephemeralOwner` было ненулевым: `0x10000174ec20006`. Существование узла и владельца дополнительно зафиксировано отдельным клиентом Python.

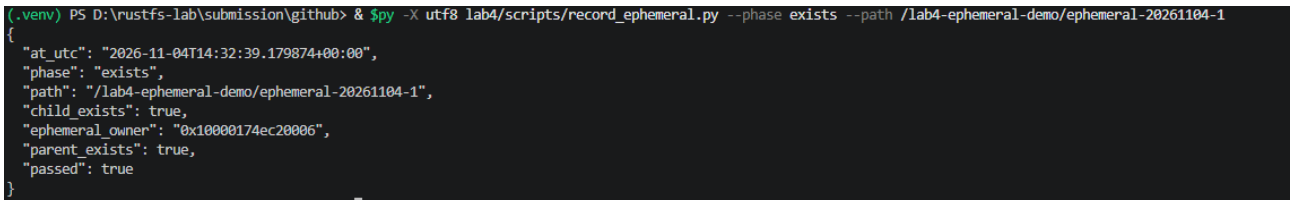


Рис. 1: Наблюдение существующего эфемерного узла и владельца его сессии.

Затем закрыл создавшую узел сессию A командой `quit`. В журнале зафиксировано закрытие сессии `0x10000174ec20006`. В оставшемся клиенте B повторная команда `get -s` вернула `Node does not exist`.

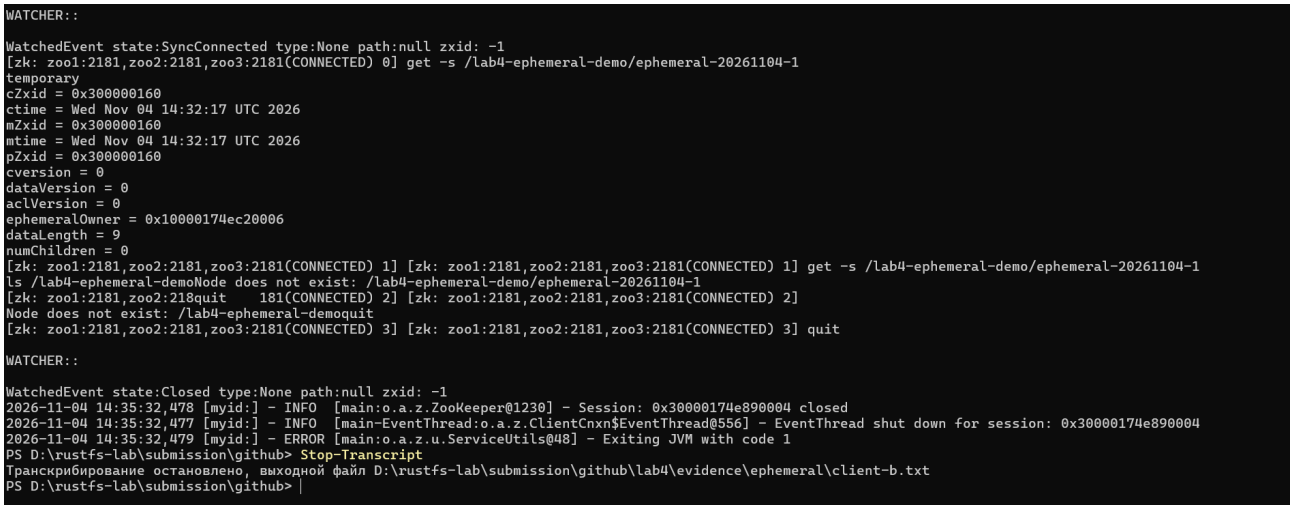


Рис. 2: Клиент B: узел сначала существует, а после закрытия сессии A отсутствует.

В конце интерактивного журнала команда `ls` была ошибочно склеена с `quit`; эта строка не использовалась как проверка родителя. Сохранение постоянного родительского узла проверено отдельно через ZooKeeper API:


```
{
  "at_utc": "2026-11-04T14:37:38.962979+00:00",
  "phase": "gone",
  "path": "/lab4-ephemeral-demo/ephemeral-20261104-1",
  "child_exists": false,
  "ephemeral_owner": null,
  "parent_exists": true,
  "passed": true
}
```


Таким образом, эфемерный узел не удалялся вручную: он исчез при закрытии создавшей сессии. При аварийном обрыве клиента удаление произошло бы после истечения согласованного тайм-аута.

4 Доработка координатора расчёта из ДЗ № 3

Вычисления выполняются на том же варианте «аналитики»: 2 057 вакансий. Использована точная локальная копия объекта

`s3://hh-lake/raw/analysts/aa4d3566148d73b7/vacancies.jsonl`

Её SHA-256 проверяется перед каждым запуском. Во время опытов сетевое чтение S3 не выполнялось; это отделяет проверку ZooKeeper от доступности RustFS.


```
{
  "input_records": 2057,
  "input_bytes": 11858115,
  "input_sha256": "aa4d3566148d73b7accc2ed97b3a3c3a63f66c873e2ee4e6dc56dff5d548f9f6",
  "reference_sha256": "3d1559ccc0967058a1c73fa7c94e9b0623b027c57e28f20ba142b29189c788f7"
}
```


Правила подсчёта не менялись: используется функция `map_batch` из № 3, удаляются повторяющиеся навыки внутри вакансии, сохраняется исходный порядок выбора навыков, строятся неупорядоченные сочетания размеров 1, 2 и 3. Для итоговых опытов выбран полный стек. Работают два процесса мапперов; одно задание содержит до 128 вакансий, окно - до четырёх заданий.

4.1 Лидерство двух координаторов

Координатор A первым создаёт эфемерный узел `/leader` в пространстве своего запуска. Координатор B ожидает освобождения узла. Новый срок лидерства увеличивает значение постоянной эпохи с проверкой версии. При истечении сессии A его эфемерный узел исчезает, B получает лидерство и новую эпоху.

Роль лидера приложения отличается от роли лидера ансамбля ZooKeeper: в опытах приостанавливался контейнер координатора A, а не сервер ZooKeeper. В обеих копиях запрошен тайм-аут 10 000 мс. Согласованное значение взято из ответа `Connect` сервера и сохранено в журналах; оно также равно 10 000 мс.

4.2 Сохранение и загрузка чекпоинта

В № 3 восстанавливался упавший маппер, но состояние координатора оставалось в памяти. Для № 4 добавлен долговечный чекпоинт: накопленные частоты, число обработанных вакансий, номер следующего задания, контрольные счётчики и параметры входа.

Чекпоинт фиксируется после успешного завершения всего окна. JSON-снимок сначала записывается в общий каталог на D: и сбрасывается на диск; затем в ZooKeeper публикуется небольшой указатель с SHA-256. При восстановлении проверяются хеш, формат и параметры расчёта. Подтверждённые задания пропускаются без повторного добавления частот.

Общий каталог `lab4/.runtime/state/` подключён к обоим координаторам. В ZooKeeper не помещаются большие таблицы частот. Такое хранение переживает остановку процесса координатора, но не защищает от отказа общего диска; этот отказ не моделировался.

В обоих итоговых опытах B загрузил чекпоинт после **512 вакансий**, продолжил с задания № 4 и посчитал оставшиеся 1 545 вакансий. SHA-256 загруженного снимка совпал с хешем сохранённого A снимка.

4.3 Методика приостановки

В № 3 медиана расчёта полного стека составляла около 0,486 с. Чтобы успеть наблюдать отказ, перед обработкой каждого окна добавлена демонстрационная задержка 3 с. Она меняет длительность, но не набор данных и не частоты.

После первого подтверждённого чекпоинта и начала следующего окна контейнер A приостанавливался командой `docker pause`. При этом останавливались все его процессы и heartbeat клиента. Обычный `sleep` только в вычислительном потоке оставил бы сессию живой и не заменил бы этот опыт.

5 Опыт без защиты финальной публикации

Идентификатор запуска: `unsafe-20261104T142800Z`. До начала опыта создан пустой постоянный указатель `/result` с `dataVersion=0`; прогресс хранится в отдельном узле. После приостановки A его сессия истекла, B получил эпоху 2, восстановился с чекпоинта и опубликовал результат. До возобновления A указатель принадлежал B и имел версию 1.

Пауза A длилась 40,266 с. После возобновления прежняя попытка A закончила оставшийся расчёт со своей эпохой 1. В этом режиме финальная запись выполнялась без проверки версии:

```
zk.set(result_path, result_pointer, version=-1)
```

Обе операции завершились успешно. В журналах есть две строки «записал результат»: сначала от B, затем от A.

Автор	Время, МСК	Сообщение	Версия
B	17:28:32.815	записал результат	1
A	17:28:57.258	записал результат	2

Фрагменты журналов: в событии загрузки поля `checkpoint_coordinator_id` и `checkpoint_epoch` относятся к создавшему снимок A. Загрузчик B установлен по файлу `coordinator-B.jsonl`; его идентификатор и эпоха указаны отдельно. Выдержки содержат выбранные поля времени, события и версии указателя:

```
{"at_utc": "2026-11-04T14:28:02.005503+00:00", "coordinator_id": "A", "event": "
    became_leader", "epoch": 1}
{"at_utc": "2026-11-04T14:28:05.483951+00:00", "coordinator_id": "A", "event": "
    checkpoint_saved", "epoch": 1, "records": 512, "next_batch": 4}
{"at_utc": "2026-11-04T14:28:17.308587+00:00", "coordinator_id": "B", "event": "
    became_leader", "epoch": 2}
{"at_utc": "2026-11-04T14:28:17.389549+00:00", "coordinator_id": "B", "event": "
    checkpoint_loaded", "epoch": 2, "records": 512, "next_batch": 4, "
    checkpoint_coordinator_id": "A", "checkpoint_epoch": 1, "source_log": "coordinator-B.
    jsonl"}
{"at_utc": "2026-11-04T14:28:32.815770+00:00", "coordinator_id": "B", "event": "
    result_published", "epoch": 2, "records": 2057, "result_version": 1}
{"at_utc": "2026-11-04T14:28:57.258644+00:00", "coordinator_id": "A", "event": "
    result_published", "epoch": 1, "records": 2057, "result_version": 2}
```

Итоговый указатель принадлежит A, его `dataVersion=2`. Численные результаты обеих попыток совпали с № 3, но старая попытка повторно опубликовала результат после потери полномочий. Одинаковая арифметика не устраняет ошибку повторной публикации.

6 Опыт с защитой по эпохе

Идентификатор запуска: `fenced-20261104T142926Z`. Для него использовано отдельное пространство ZooKeeper и заново созданный указатель версии 0. Вход, правила расчёта,

тайм-аут и точка паузы сохранены такими же.

При публикации проверка версии эпохи и обновление указателя объединены одной атомарной транзакцией:

```
transaction = zk.transaction()
transaction.check(epoch_path, version=old_epoch_version)
transaction.set_data(result_path, result_pointer)
results = transaction.commit()
```

После истечения сессии A запасной B получил новую эпоху, загрузил частичный чекпоинт и закончил расчёт. Пауза A составила 40,203 с. Возобновившийся A сохранил старый токен и попытался опубликовать результат. Сервер отклонил транзакцию с **BadVersionError**; указатель B остался версии 1.

```
{"at_utc": "2026-11-04T14:29:27.431798+00:00", "coordinator_id": "A", "event": "
    became_leader", "epoch": 1}
{"at_utc": "2026-11-04T14:29:30.884031+00:00", "coordinator_id": "A", "event": "
    checkpoint_saved", "epoch": 1, "records": 512, "next_batch": 4}
{"at_utc": "2026-11-04T14:29:43.329393+00:00", "coordinator_id": "B", "event": "
    became_leader", "epoch": 2}
{"at_utc": "2026-11-04T14:29:43.546230+00:00", "coordinator_id": "B", "event": "
    checkpoint_loaded", "epoch": 2, "records": 512, "next_batch": 4, "
    checkpoint_coordinator_id": "A", "checkpoint_epoch": 1, "source_log": "coordinator-B.
    jsonl"}
{"at_utc": "2026-11-04T14:29:59.235061+00:00", "coordinator_id": "B", "event": "
    result_published", "epoch": 2, "records": 2057, "result_version": 1}
{"at_utc": "2026-11-04T14:30:22.442445+00:00", "coordinator_id": "A", "event": "
    publication_rejected", "result_version": 1, "error": "BadVersionError", "old_epoch": 1,
    "current_epoch": 2, "result_coordinator": "B"}
```

Версия эпохи и `dataVersion` указателя - разные величины. У B эпоха равна 2, но указатель результата обновлён только один раз, поэтому его версия равна 1. Исключения в списке результатов `transaction.commit()` также проверяются явно.

В обоих режимах изменения общего указателя чекпоинта защищены эпохой. Различается финальная публикация результата. Для демонстрации устаревшая попытка после потери лидерства могла досчитать локальный результат и дойти до серверной проверки записи. Новое подключение не выдавало ей новую эпоху и не означало повторного получения лидерства. Отдельный неизменяемый файл отклонённой попытки не считается официальным результатом.

Одна блокировка не останавливает уже выполняющийся старый процесс. Проверка эпохи на границе публикации не позволяет этому процессу изменить официальный указатель после смены лидера.

7 Время переключения и проверка результатов

Время измерял внешний контроллер одними монотонными часами. Начало интервала - успешный возврат команды `docker pause`; конец - наблюдение сброшенного события `became_leader` в журнале B. Интервал опроса равен 0,1 с. Время выполнения самой команды pause сохранено отдельно.

Измеренное время включает задержку опроса и записи журнала, поэтому не объявляется точным моментом внутренней фиксации ZooKeeper. Настроенные 10 секунд не подставлялись вместо измерения.

Показатель	Без защиты	С эпохой
Смена лидера, с	11,656	12,281
Пауза A, с	40,266	40,203
Сессия, мс	10 000	10 000
Восстановлено из чекпоинта, вакансий	512	512
Принятых финальных публикаций	2	1
Финальный автор / версия указателя	A / 2	B / 1
Ответ прежнему A	Запись принята	`BadVersionError`

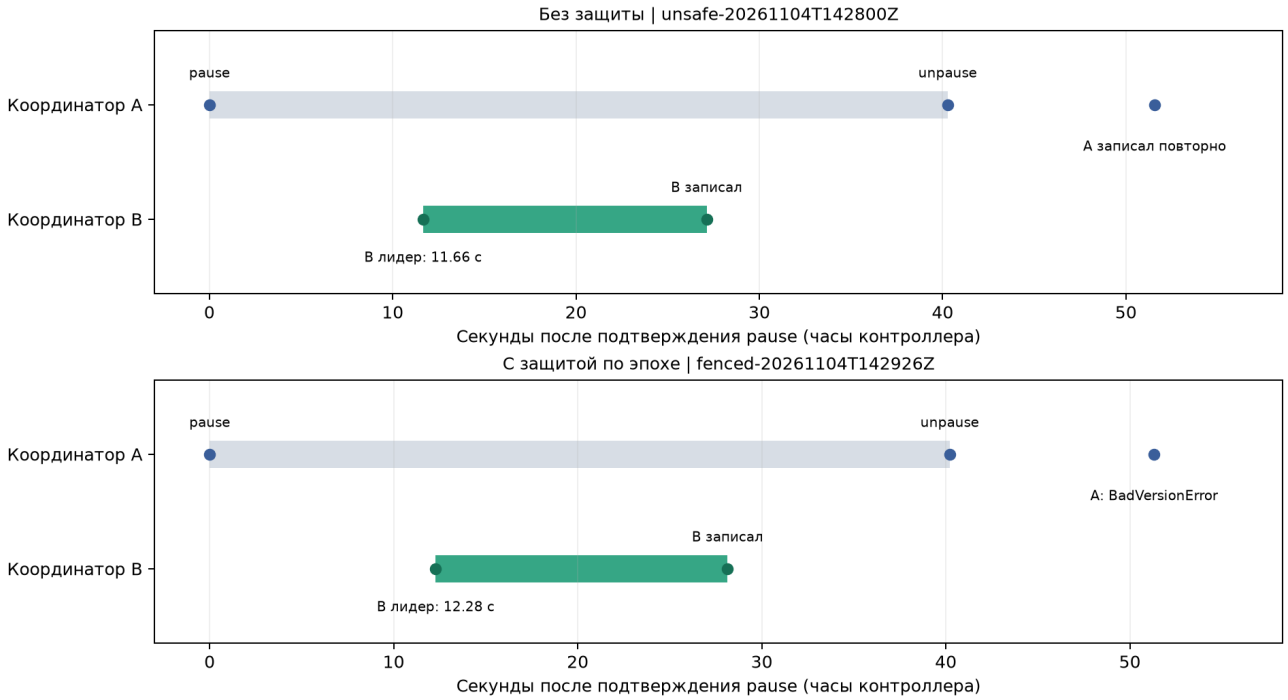


Рис. 3: Временные шкалы обоих опытов по часам внешнего контроллера.

Официальные результаты и файлы обеих попыток в каждом опыте сравнены с контрольной таблицей № 3 **по каждому ключу и частоте**, а не только по итоговой сумме:

Опыт	Попытка	Ключей	Различий	SHA-256
Без защиты	A	101 718	0	Совпадает
Без защиты	B	101 718	0	Совпадает
С эпохой	A	101 718	0	Совпадает
С эпохой	B	101 718	0	Совпадает

Контрольные числа полного стека:

Размер сочетания	Сумма частот	Уникальных ключей
Навык	8 810	1 256
Пара	40 778	15 647
Тройка	180 490	84 815

Во всех четырёх файлах результатов совпал и SHA-256 эталонного CSV:

`3d1559ccc0967058a1c73fa7c94e9b0623b027c57e28f20ba142b29189c788f7`

Дополнительно прошли шесть автономных тестов чекпоинта и два интеграционных теста на настоящем ZooKeeper. Проверены продолжение без двойного счёта, отказ при повреждённом снимке и несовпадении параметров, атомарное отклонение старой эпохи и версия 2 после двух незащищённых записей. Итог - 8 тестов без ошибок и пропусков.

8 Выводы

Ансамбль из трёх ZooKeeper продолжил работу после потери одного сервера и перестал подтверждать клиентские операции после потери двух. Эфемерный узел исчез при закрытии создавшей сессии. Запасной координатор восстановил частоты и прогресс из чекпоинта, обработал остаток данных и получил те же результаты, что в № 3.

Приостановленный процесс после возобновления может продолжить старую вычислительную попытку, хотя право лидерства уже потеряно. Без проверки версии это привело к повторной публикации и версии 2. Атомарная проверка эпохи отклонила старую запись с `BadVersionError` и сохранила результат запасного координатора. Чекпоинт обеспечивает восстановление вычислений, а эпоха отдельно защищает право публикации.

9 Ссылка на репо

Исходный код, конфигурации, журналы обоих координаторов, измерения и полные результаты сверок: `https://github.com/dmitmorozov/data-engineering-lab3/tree/main/lab4`.